

Sesión 32: Reducción de dimensionalidad y representación de variables

Laura Sánchez Fernández

ANTES DE EMPEZAR

¿Dónde estábamos?



Sesión 30

Preparación para Machine Learning

- Ciclo de EDA: preguntar, responder, refinar
- 4 dimensiones de calidad: exactitud, completitud, coherencia, frescura
- Imputación (media / mediana / moda / regresión)
- División honesta: entrenamiento / validación / prueba
- Regularización (Lasso / Ridge) y métricas más allá del Accuracy



Sesión 31

Introducción práctica al Machine Learning

- Jerarquía IA ⊃ Machine Learning ⊃ Deep Learning
- El ciclo: Datos → Modelo → Evaluación → Mejora
- 3 familias: supervisado, no supervisado, por refuerzo
- Clasificación vs. regresión
- No supervisado: Clustering (grupos) y PCA (resumir variables)

HOJA DE RUTA

Objetivos de la sesión



Diagnosticar

Por qué demasiadas variables perjudican un modelo (multicolinealidad, ruido, sobreajuste).



Representar

Cómo escalar, codificar y construir variables antes de modelar (feature engineering).



Extraer — PCA

Resumir variables correlacionadas en componentes principales, explicado en lenguaje simple.



Extraer — t-SNE

Visualizar estructura no lineal en 2D preservando relaciones de vecindad.



Comparar

Cuándo usar selección vs. extracción, y PCA vs. t-SNE, con un caso aplicado (churn real).

DATA SCIENCE UC
FACULTAD DE MATEMÁTICAS

01

El problema de la dimensionalidad

Por qué demasiadas variables perjudican y no ayudan, a un modelo.

¿Qué pasa cuando hay demasiadas variables?

Costo computacional

Más columnas implican más tiempo de procesamiento y entrenamiento.

Multicolinealidad

Variables muy correlacionadas generan inestabilidad e ineficiencia en regresiones, redes neuronales, etc.

Variables ruido

No todas las columnas aportan información real al modelo.

Sesgo vs. varianza

Con pocas observaciones, más variables aumentan la varianza de las predicciones sin compensar el sesgo.

Por qué vale la pena reducir dimensiones



Más fácil de poner en producción



Incrementa la performance del modelo



Remueve variables irrelevantes



Entrenamientos más eficientes



Más fácil de depurar



Más fácil de comprender

¿Manifold, y por dónde reducimos?

¿Qué es un manifold?

Un globo terráqueo (3D) se puede representar en un mapa plano (2D) sin perder las relaciones esenciales entre países y continentes.

Un manifold es esa estructura de menor dimensión, "doblada" dentro de un espacio mayor, que conserva la forma real de los datos. Reducir dimensionalidad es, en esencia, encontrar y "desdoblar" ese manifold.

Por eso hoy usaremos dos técnicas con lógicas distintas: PCA (lineal) y t-SNE (no lineal).

Camino 1 — Selección de variables

Eliminar columnas: quitamos variables redundantes o poco informativas y conservamos las originales (más interpretable).

Camino 2 — Creación / extracción de variables

Combinar columnas en nuevas variables (p. ej. componentes principales) que resumen la información original. Es lo que veremos hoy con PCA y t-SNE.

02

Representación de variables

Escalar, codificar y construir: cómo entregamos la información al modelo.

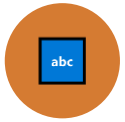
Diferenias entre Calidad de datos y representación

En la Sesión 30 nos aseguramos de que los datos fueran exactos, completos, coherentes y frescos. Eso responde "¿son datos confiables?". Hoy respondemos una pregunta distinta: "¿en qué forma se las entrego al modelo?"



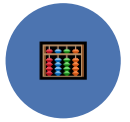
Escalas distintas

"segundos" llega a miles y "reclamos" es 0 o 1. Sin ajustar, la variable de mayor magnitud domina distancias y componentes.



Texto y categorías

Un modelo no entiende "prepago" o "plan": necesita una representación numérica de esas categorías.



Información implícita

A veces el dato útil no es una columna cruda, sino una combinación de columnas (una razón, una tasa, un promedio).

Idea clave: PCA y t-SNE trabajan con distancias y varianzas. Si las variables no están bien representadas, el algoritmo "resume" ruido, no información.

Escalamiento y estandarización

Antes de aplicar PCA, t-SNE o cualquier método basado en distancias, las variables numéricas deben quedar en una escala comparable — como convertir todo a la misma unidad antes de comparar.

Estandarización (z-score)

Mide cada valor en "cuántas desviaciones típicas se aleja del promedio". Todas las variables terminan centradas en 0. Es el estándar para PCA y t-SNE.

Normalización (min-max)

Comprime todos los valores para que queden entre 0 y 1, conservando el orden. Útil cuando se necesitan límites fijos (p. ej. redes neuronales).

En Python: StandardScaler() y MinMaxScaler() de sklearn.preprocessing — siempre ajustados (fit) solo con el conjunto de entrenamiento (Sesión 30: evitar Data Leakage).

Codificación de variables categóricas

Ordinal encoding

Nominal → entero cuando existe un orden real.

```
monto: 0 (bajo) … 9 (alto)
```

One-hot encoding

Una columna binaria por categoría; evita imponer un orden falso.

```
tipo_plan → tipo_plan_prepago, tipo_plan_plan
```

Target / frequency encoding

Reemplaza la categoría por un estadístico (tasa de fuga, frecuencia). Útil con muchas categorías.

```
comuna → tasa de fuga histórica de esa comuna
```

En nuestro dataset churn: tipo_plan y estado son categóricas codificadas como enteros (1/2) — funcionan directo en modelos de árbol, pero conviene one-hot para PCA/t-SNE porque estos asumen una noción de distancia numérica continua.

Creación de variables (feature engineering)

A veces la variable más informativa no existe todavía en el dataset: se construye combinando columnas.

duracion_prom_llamada	`segundos ÷ frec_llamadas`	Un cliente con muchas llamadas cortas se comporta distinto de uno con pocas llamadas largas.
sms_por_antiguedad	`frec_sms ÷ antiguedad`	Normaliza el uso de SMS por el tiempo que lleva el cliente en la compañía.
intensidad_contacto	`n_contactos ÷ frec_llamadas`	Aproxima cuán concentrada está la actividad del cliente en pocos contactos.

Estas variables nuevas se agregan al conjunto ANTES de escalar y reducir dimensionalidad: la representación es un paso previo, no un sustituto de PCA/t-SNE.

03

Extracción de variables — PCA

De muchas variables correlacionadas a componentes que resumen lo esencial.

Análisis de Componentes Principales

En una frase: PCA toma un montón de variables y las "resume" en unas pocas nuevas variables (los componentes), tratando de perder la menor cantidad de información posible.

Cada componente es como una ***mezcla o receta*** hecha con un poco de cada variable original. El primer componente es la mezcla que mejor captura el patrón principal de los datos; el segundo, el siguiente patrón más importante que no se repite con el primero; y así sucesivamente.

Objetivo: explicar la mayor parte de la variabilidad de los datos usando muchas menos variables que el conjunto original.



No supervisado

No usa la variable objetivo (fuga) para construir los componentes.



Requiere escala

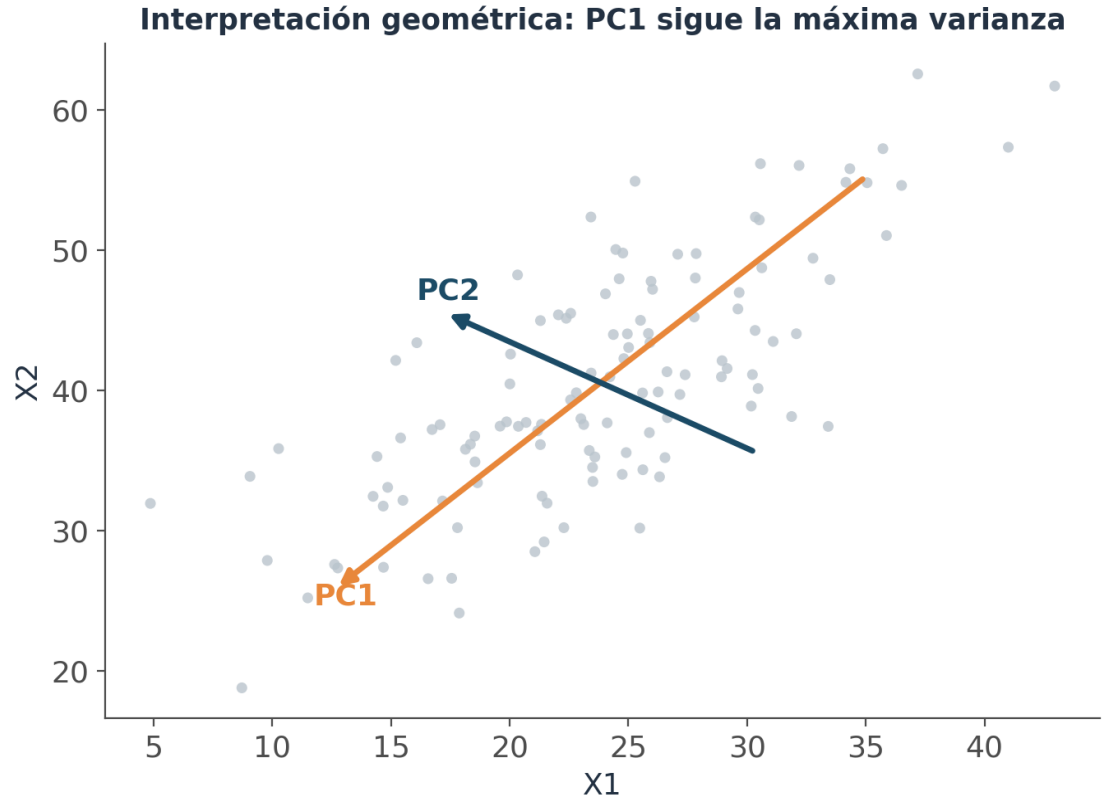
Se calcula sobre variables estandarizadas (ver Parte 2).



Es una "receta"

Cada componente combina un poco de cada variable original — el detalle matemático lo hace el computador por ti.

La intuición detrás de PCA



No necesitas el álgebra para usar PCA — basta con entender la idea:

1. PCA busca la dirección donde los datos varían más. Ahí es donde hay más información para resumir.

2. Esa dirección se convierte en el primer componente (PC1, la línea naranja). La siguiente dirección más informativa, que no repite lo que ya explicó PC1, es el segundo componente (PC2, la línea azul).

3. Con muchas variables, este mismo razonamiento se repite: cada nuevo componente capta el patrón más fuerte que aún no se había explicado.

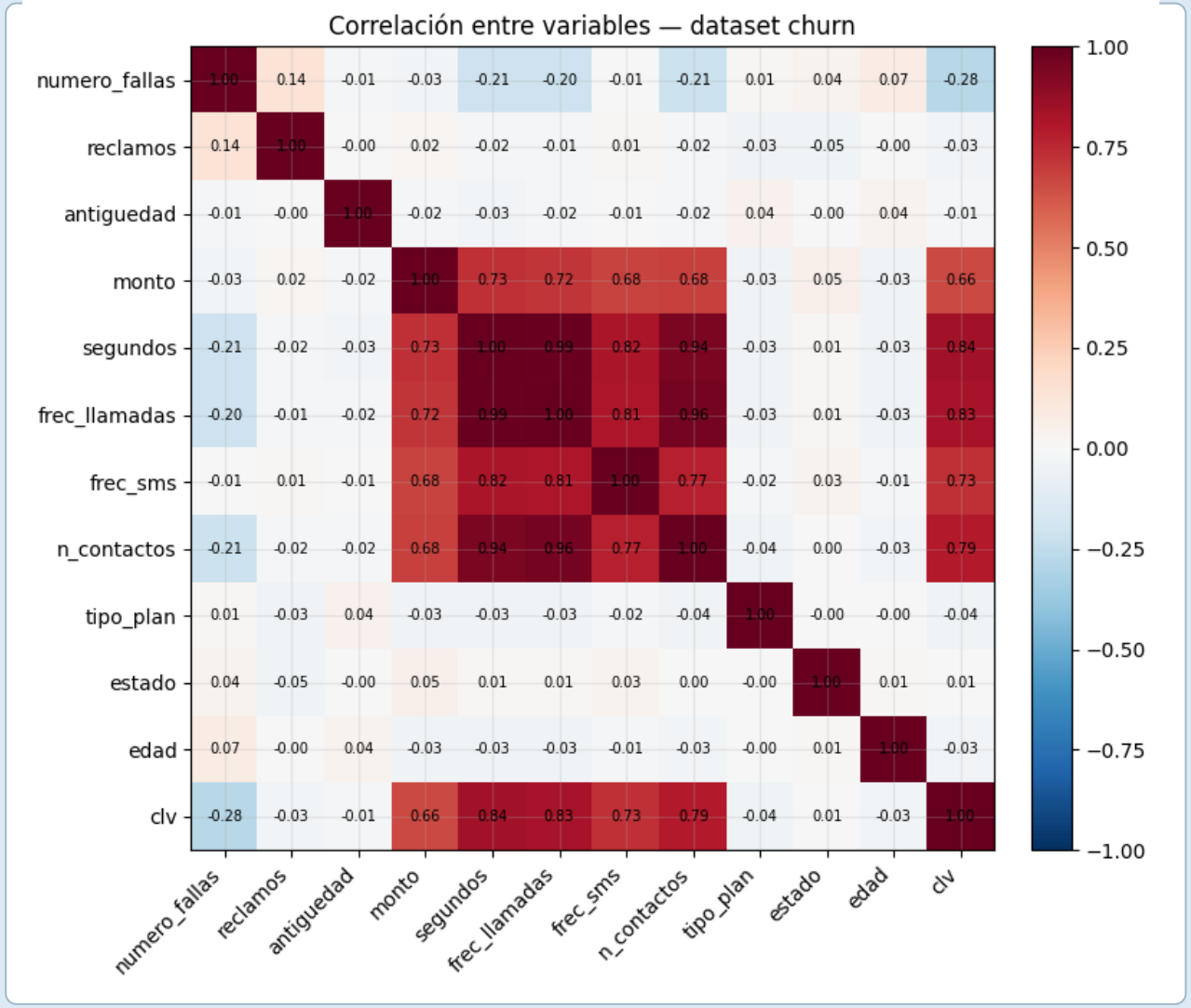
Aplicación: el dataset real de churn

Retomamos el ejemplo de la Sesión 31 ("¿el cliente cancelará su suscripción?"), ahora con los datos reales de comportamiento de 1500 clientes de una telco.

`numero_fallas`	N° de llamadas fallidas	`frec_sms`	Total de mensajes de texto
`reclamos`	0: sin reclamo · 1: con reclamo	`n_contactos`	N° de números distintos contactados
`antiguedad`	Meses en suscripción	`tipo_plan`	1: prepago · 2: plan
`monto`	Nivel del plan (0 bajo … 9 alto)	`estado`	1: activa · 2: no activa
`segundos`	Segundos totales en llamadas	`edad`	Edad del cliente
`frec_llamadas`	Total de llamadas	`clv`	Valor del cliente (Customer Lifetime Value)

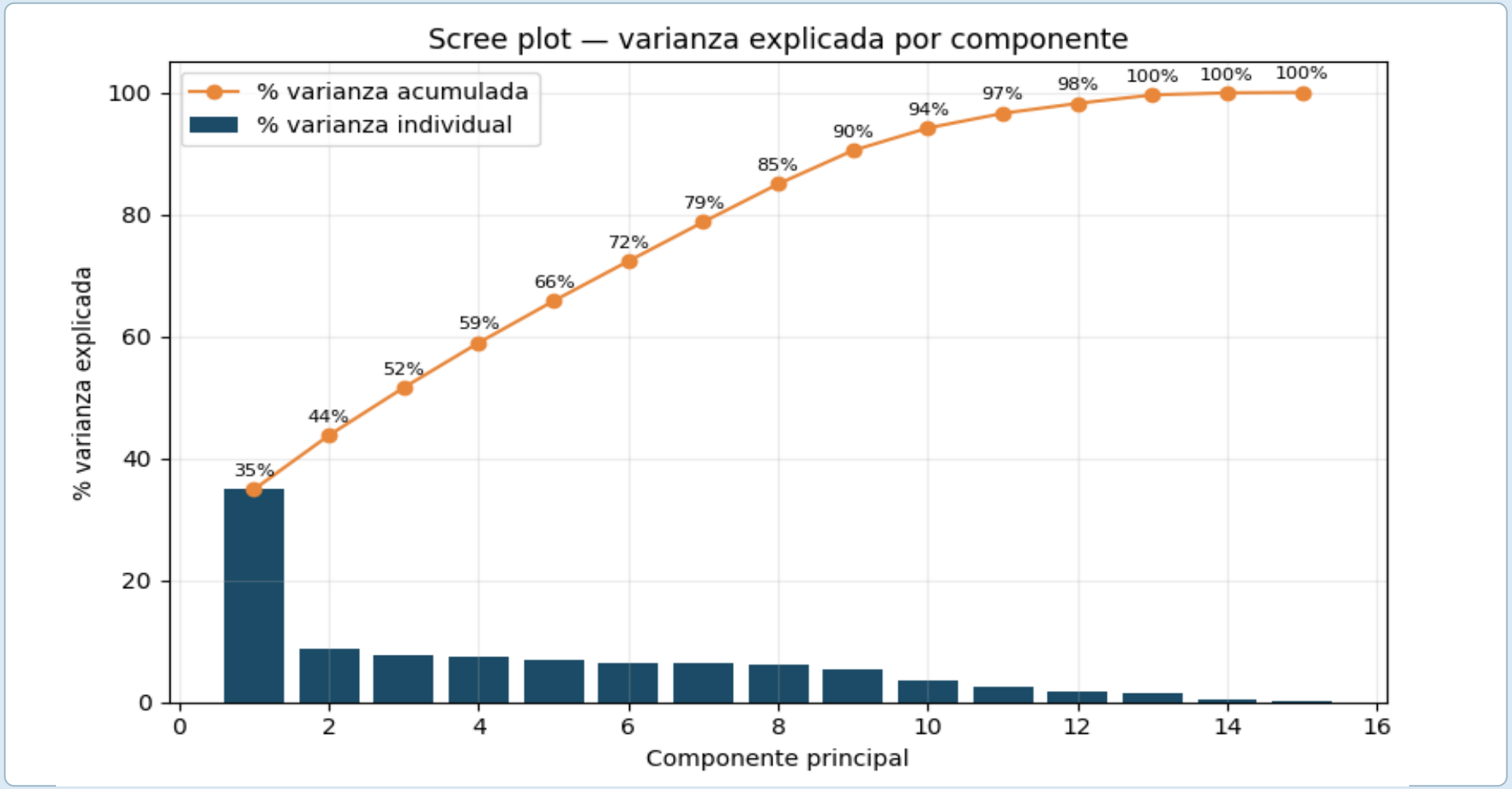
Variable objetivo: fuga (1 = abandonó, 0 = activo). Hoy la dejamos fuera del PCA — solo la usamos después, para colorear y evaluar — porque PCA es no supervisado.
Antes de este análisis, corregimos un par de cosas típicas de la Sesión 30: valores de edad imposibles (negativos o sobre 100) y datos faltantes en varias columnas.

Antes de reducir: revisar correlaciones

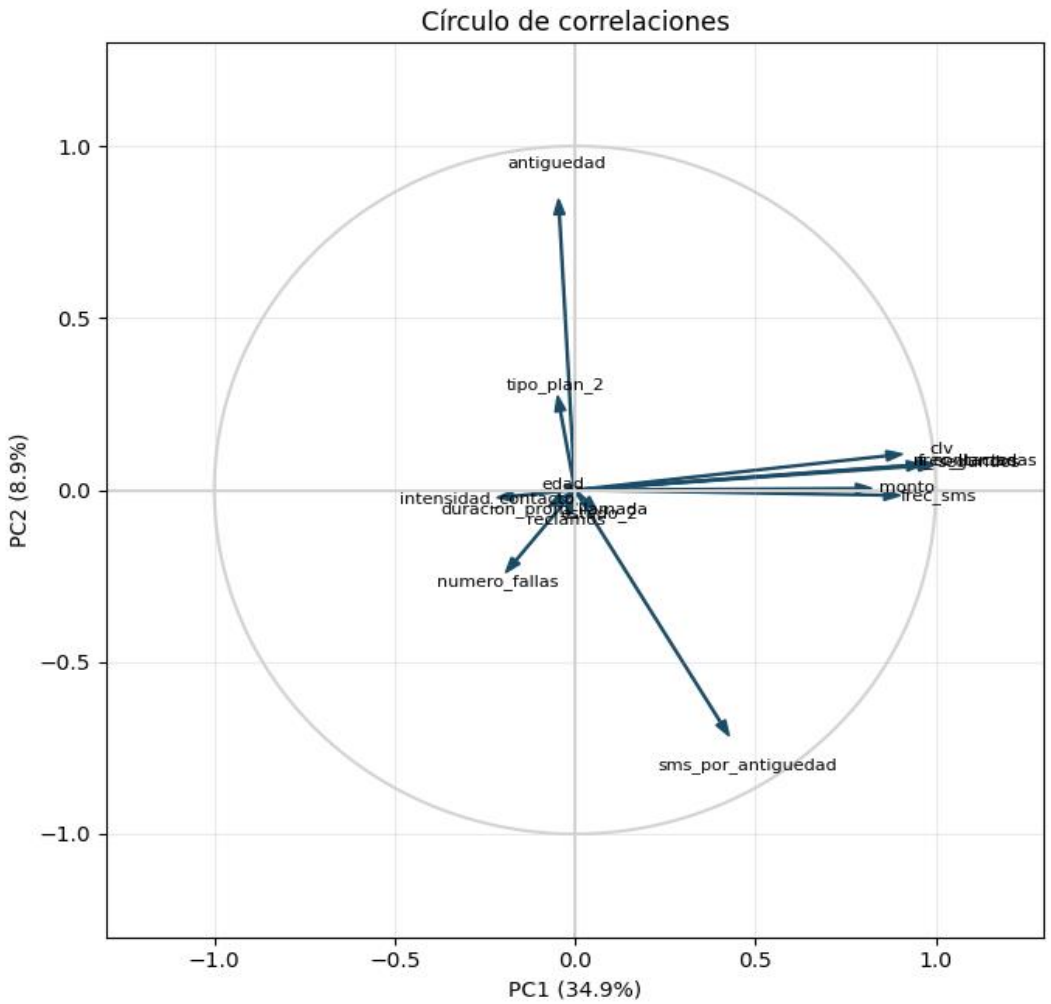


¿Cuántas componentes elegimos?

Cada barra es "cuánta información nueva" aporta ese componente. La línea naranja es la suma acumulada. Buscamos el punto donde agregar un componente más ya no suma casi nada — ahí nos detenemos.



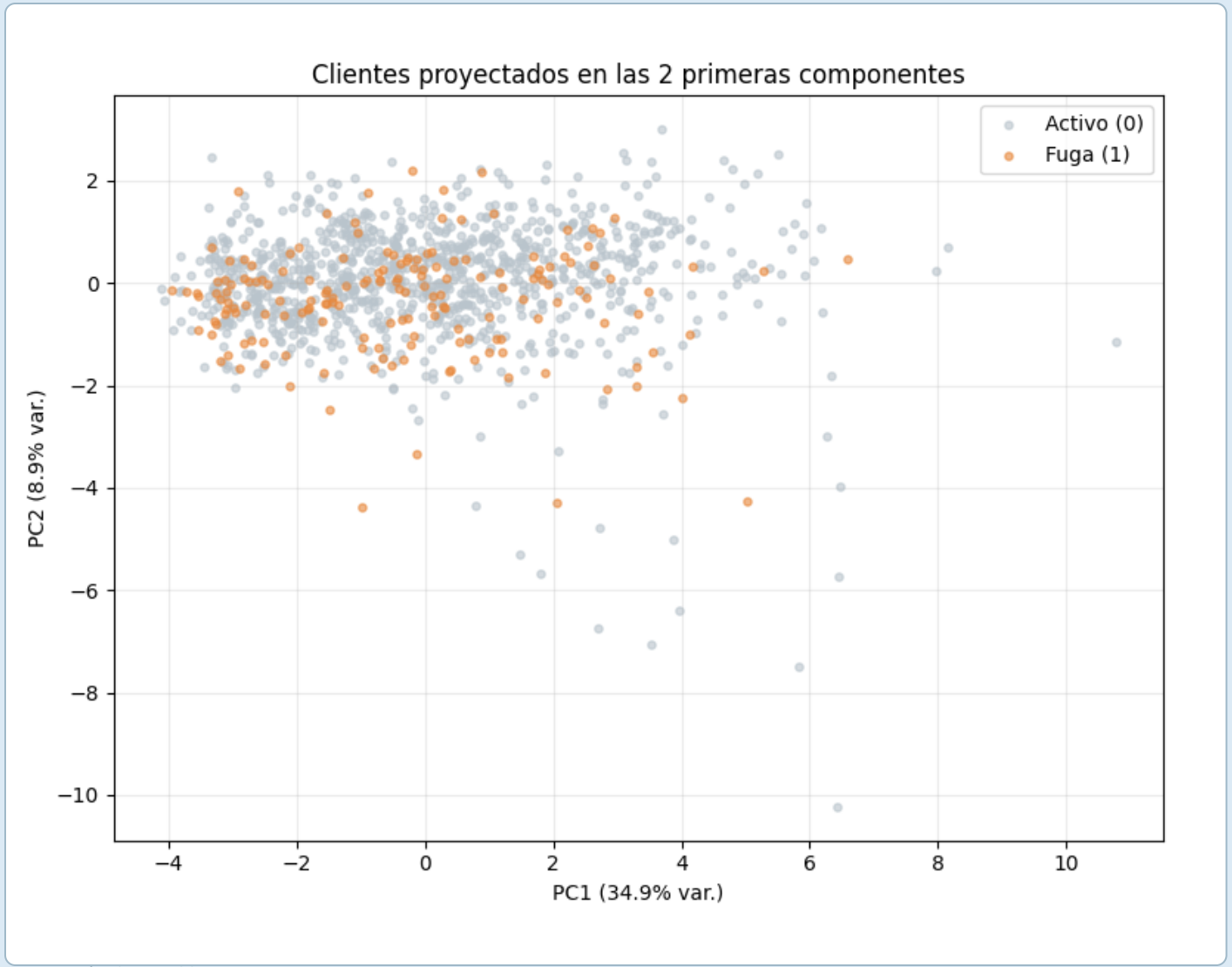
¿Qué variables arma cada componente?



PC1 agrupa el uso general del servicio (segundos, llamadas, contactos, SMS, monto, CLV): mientras más usa el cliente el servicio, más alto es su PC1.

PC2 mezcla número de fallas técnicas y reclamos: un valor alto en PC2 se parece a un cliente con más fricción/insatisfacción con el servicio.

Proyectando clientes en 2 dimensiones



¿Reducir dimensiones cuesta desempeño?

Comparamos una regresión logística entrenada con las 15 variables (ya representadas: escaladas + codificadas) vs. con solo 7 componentes principales (≈80% de la varianza).

Modelo	N° variables	F1-score	MCC
Variables originales	15	0.270	0.265
8 componentes (PCA)	8	0.267	0.252

Lectura: con menos de la mitad de las columnas (8 en vez de 15), el desempeño baja apenas un poco. Ese es el trade-off real de PCA: normalmente se cede algo de desempeño a cambio de un modelo mucho más simple, liviano y fácil de mantener. Si esa pequeña pérdida vale la pena depende del problema — no siempre es gratis, pero tampoco suele ser caro.

04

t-SNE

Un mapa no lineal para explorar la estructura de los datos.

Una alternativa no lineal: t-SNE

t-Distributed Stochastic Neighbor Embedding (Hinton & van der Maaten, 2008) busca un ***embedding***: un mapa en 2 o 3 dimensiones que mantiene juntos a los puntos que eran parecidos originalmente, y separados a los que no. A diferencia de PCA, no le importa la varianza total, sino "quién es vecino de quién".



La idea

Si dos clientes se parecen mucho en las variables originales, t-SNE los va a dibujar cerca en el mapa 2D. Si son distintos, los aleja.



Embedding

Es solo el nombre técnico de "mapa simplificado": una representación más chica que conserva las relaciones importantes entre los datos.



No lineal

A diferencia de PCA, no asume que la relación entre variables sea una simple mezcla lineal — puede capturar patrones más retorcidos.

¿Cómo funciona?

1

Mide las "vecindades" originales

Para cada cliente, t-SNE calcula qué tan probable es que cada otro cliente sea su "vecino", usando las variables originales (con muchas columnas).

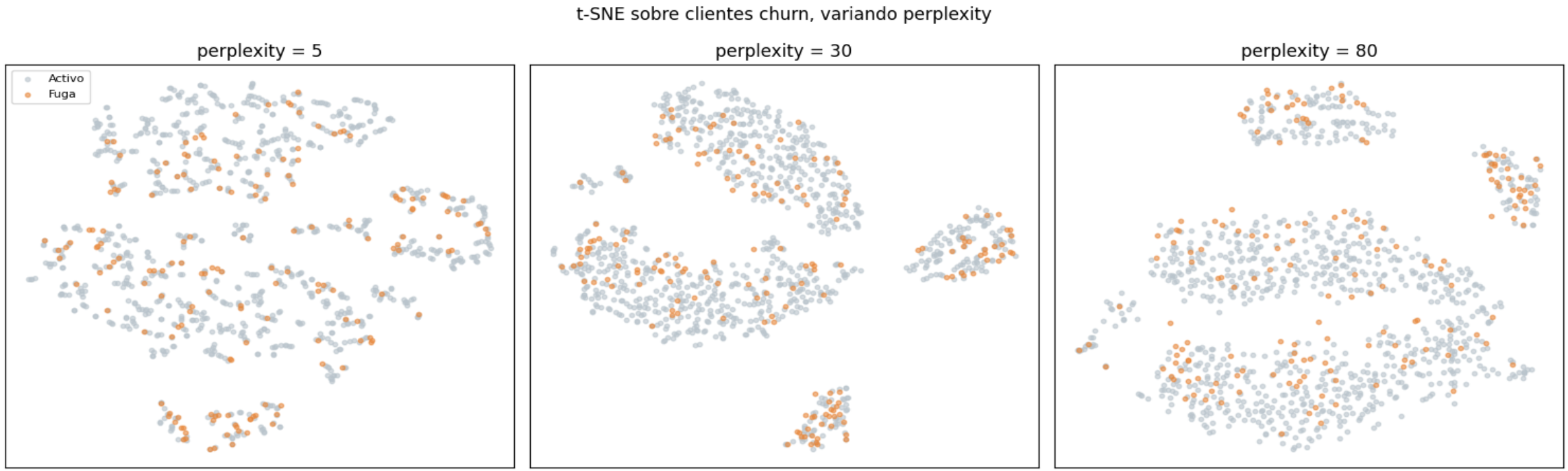
2

Dibuja un mapa en 2D que respete esas vecindades

Ubica a cada cliente como un punto en el plano, y va moviendo esos puntos de a poco hasta que las vecindades del mapa 2D se parezcan lo más posible a las vecindades originales.

Sin fórmulas: el algoritmo va "acomodando" los puntos en el mapa, un poquito en cada vuelta, hasta que el resultado se parece lo más posible al original. Es un proceso de prueba y ajuste, no una fórmula que se resuelve de una vez — por eso puede tardar y por eso el resultado varía un poco entre corridas.

El hiperparámetro clave: perplexity



En lenguaje coloquial: **perplexity** es más o menos "a cuántos vecinos le presto atención" al armar el mapa. **Baja** → grupos chicos y muy locales. **Alta** → una vista más panorámica, con grupos más grandes. En la práctica se prueba entre 5 y 50.

Consideraciones prácticas



Perplexity acotado

Debe ser menor al número de observaciones; en la práctica se prueba entre 5 y 50.



No determinístico

Cada corrida puede variar levemente el resultado (fijar random_state ayuda a reproducir).



No es un modelo de clúster

Expande grupos densos y compacta los dispersos: no interpretar distancias entre clústeres como "distancia real".



Costo computacional alto

Comparar todas las parejas de observaciones crece rápido con el número de datos.

PCA vs. t-SNE, y otras alternativas

PCA

Combina variables de forma simple (lineal) · conserva la variabilidad global · da el mismo resultado cada vez · rápido · sirve para producción.

t-SNE

Combina variables de forma más compleja (no lineal) · conserva vecindades locales · el resultado varía un poco cada vez · más lento · pensado para visualizar, no para producción.

Otros algoritmos que existen en la misma familia

Simples (lineales): Análisis Factorial · Análisis Discriminante · SVD

Más complejos (no lineales): Autoencoders · IsoMap · Local Linear Embedding · UMAP

De la calidad de los datos a variables listas para modelar

30

Calidad de datos

Diagnosticamos, imputamos y dividimos honestamente el dataset.

›

31

Aprendizaje

Entendimos el ciclo de ML y las familias supervisado / no supervisado.

›

32

Representación y reducción

Escalamos, codificamos y resumimos variables con PCA / t-SNE.

De datos confiables, a un modelo que aprende, a variables bien representadas y resumidas: cada sesión resuelve una condición para que el modelo funcione en la vida real.

Para pensar después de la clase

- De las variables de su proyecto, ¿cuáles cree que están altamente correlacionadas entre sí?
- ¿Qué variable nueva podría construir combinando dos columnas que ya tiene (feature engineering)?
- Si tuviera que quedarse con la mitad de sus variables, ¿las eliminaría (selección) o las resumiría con PCA (extracción)?
- ¿Cuándo preferiría un mapa t-SNE para explorar sus datos, y cuándo un PCA para reducir antes de modelar?